

LAB 7

Developing AI Intuition Using LLMs and Prompt Engineering

Unit 7: LLMs and Prompt Engineering

Week of July 20

Agenda

15 min

Welcome + Lab Opener

40 min

Interactive Concept Overview

30 min

Business Understanding Session

10 min

Break

70 min

Coding Lab Working Session

10 min

Group Retro + Closing

5 min

Lab Survey

15
MIN

Lab Opener

Who Wrote This?

Overview

A prompt about the 2026 FIFA World Cup was given to one human and two AI models. Your job: figure out who wrote which response.

The prompt: "Who were you rooting for in this year's World Cup from the start? Answer as if you are my friend trying to convince me on who to root for. Answer in less than 150 words."

INSTRUCTIONS

- 1. The next slide shows the three responses labeled A, B, and C. Read all three.*
- 2. Drop your answers in the chat:*
 - Human = [A, B, or C]*
 - AI model 1 = [A, B, or C]*
 - AI model 2 = [A, B, or C]*
- 3. After I reveal which one was which, we will discuss: what gave away the AI generated models for you or what fooled you?*

Which answers were AI-generated and which one was written by a human?

Prompt: "Who were you rooting for in this year's World Cup from the start? Answer as if you are my friend trying to convince me on who to root for. Answer in less than 150 words."

A

I was rooting for Uzbekistan because that's where I was born and this is their first time qualifying for the World Cup. Actually, this is the first time a central Asian country has qualified for the World Cup. They are a hard-working team and it's always great to root for the rookie, especially one that's from a region that typically doesn't have any representation in the World Cup. Even though it's unlikely that they'll advance far into the competition, the fact that they're there is already encouraging the next generation of soccer players in Uzbekistan.

B

Cape Verde, no question. A tiny island nation, population under 600,000, playing in their first-ever World Cup, and they came in and made a real run at this thing. That's the story that made the World Cup worth watching -- the underdog nobody circled on their bracket showed up and made the giants sweat. It's not just a nice story either, they backed it up on the field. If you were rooting for Cape Verde, you got chaos, joy, and serious bragging rights every time they pulled off another shock. That was my pick.

C

I was all in on Morocco national football team from day one. Come on, how could you not have been? They're fearless, they defend like every tackle means something, and every deep run feels like they're making history all over again. They're the kind of team that makes neutral fans care because they never look intimidated, no matter who's across from them. If I were to emotionally invest in a team from the beginning, I would have recommended picking one that fights for every inch and gives you a reason to yell at your TV. Trust me, Morocco is a fun ride.  

Which answers were AI-generated and which one was written by a human?

Prompt: "Who were you rooting for in this year's World Cup from the start? Answer as if you are my friend trying to convince me on who to root for. Answer in less than 150 words."

A

I was rooting for Uzbekistan because that's where I was born and this is their first time qualifying for the World Cup. Actually, this is the first time a central Asian country has qualified for the World Cup. They are a hard-working team and it's always great to root for the rookie, especially one that's from a region that typically doesn't have any representation in the World Cup. Even though it's unlikely that they'll advance far into the competition, the fact that they're there is already encouraging the next generation of soccer players in Uzbekistan.

Human

B

Cape Verde, no question. A tiny island nation, population under 600,000, playing in their first-ever World Cup, and they came in and made a real run at this thing. That's the story that made the World Cup worth watching -- the underdog nobody circled on their bracket showed up and made the giants sweat. It's not just a nice story either, they backed it up on the field. If you were rooting for Cape Verde, you got chaos, joy, and serious bragging rights every time they pulled off another shock. That was my pick.

Claude

C

I was all in on the Morocco national football team from day one. Come on, how could you not have been? They're fearless, they defend like every tackle means something, and every deep run feels like they're making history all over again. They're the kind of team that makes neutral fans care because they never look intimidated, no matter who's across from them. If I were to emotionally invest in a team from the beginning, I would have recommended picking one that fights for every inch and gives you a reason to yell at your TV. Trust me, Morocco is a fun ride.  

Gemini

Before we dive in — a quick review of lab 6

- Most students completed the core Keras workflow correctly.
- Common issue: missing validation setup, causing incomplete metrics or plot errors.
- Test accuracy was typically 80% to 83%.
- Most students identified overfitting, but analysis quality varied.
- Main improvement area: fully run all cells and support conclusions with results.

40
MIN

Interactive Concept Overview

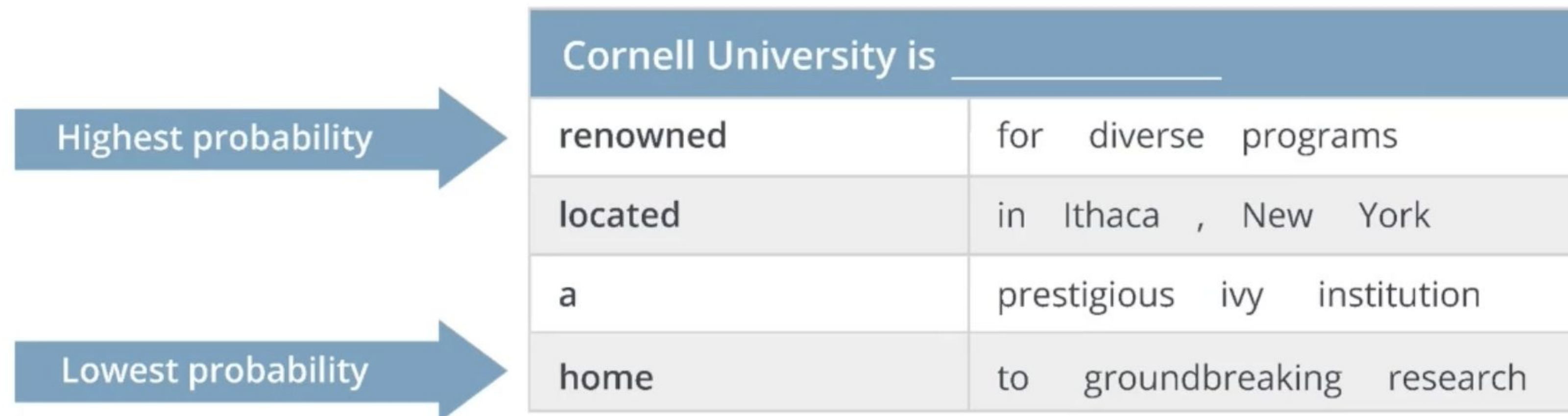
Before we dive in — a quick question



Drop in the chat: What is one thing from this week's async content you feel solid on — and one thing you're still fuzzy on?

How LLMs Generate Text

An LLM predicts the next most likely token based on patterns learned from massive text corpora. It does not look up facts or retrieve stored knowledge -- it completes sequences.



Temperature controls how the model chooses its response. At temperature 0, it always picks the highest-probability output. At higher temperatures, it samples from lower-probability tokens, producing more varied and creative, but also less reliable, output.

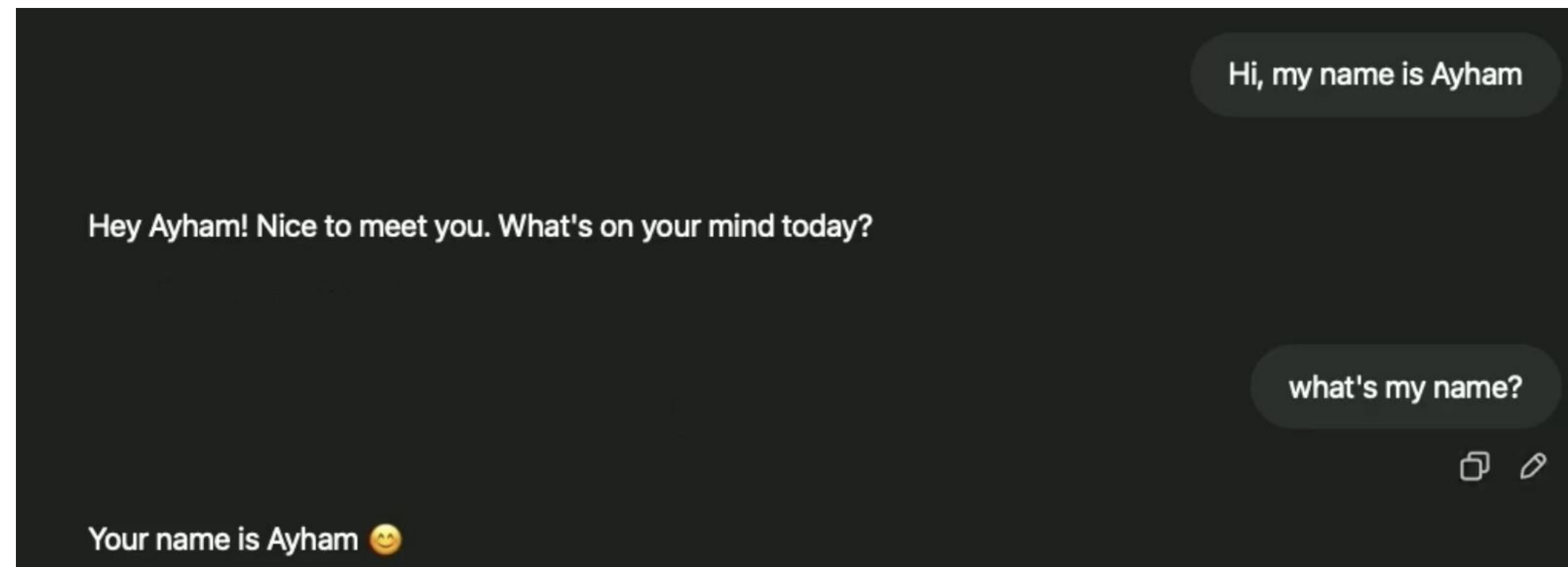
KEY TAKEAWAY

Because LLMs complete patterns rather than retrieve facts, they can produce fluent, confident, and wrong output -- which is where limitations like hallucinations come from.

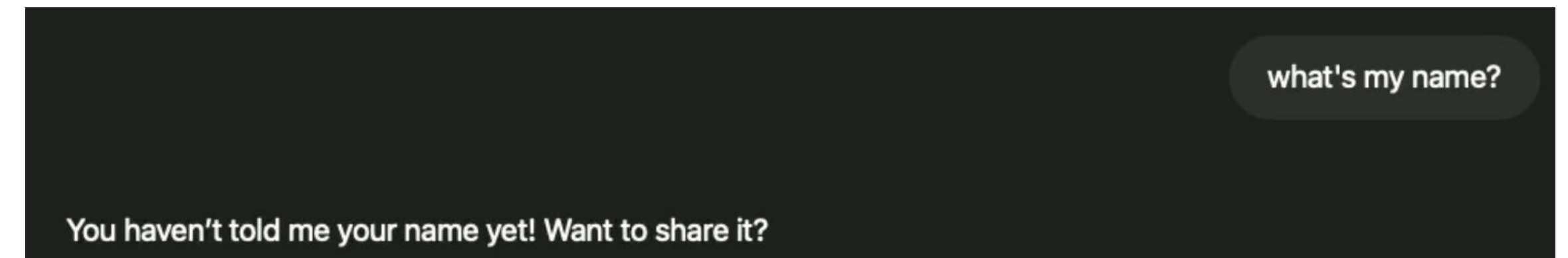
Context Window and Memory Limitations

LLMs have no persistent memory between conversations. Each new session typically starts completely fresh. The model cannot recall anything from a previous chat unless that history is explicitly included in the new prompt.

Session 1



Session 2



LLM Limitations: Hallucinations

LLMs generate text by pattern completion, not by fact retrieval. This means they can produce confident, fluent, and entirely wrong output. We call this hallucinations.

Prompt

"Digitize these meal receipts and generate an expense report."

Response

RECEIPT

Lunch 45.60

Subtotal 45.60

Tax 3.65

Total49.25

RECEIPT

Lunch 103.27

Subtotal 103.27

Tax 8.26

Total111.53

Expense Report	
Lunch	
Sandwich	19.50
Chips	5.25
Fruit plate	12.50
Soda	8.35
Subtotal	45.60
Tax	3.65
Lunch Total	49.25
Dinner	
Steak	49.98
Salad	18.75
Beer	18.95
Dessert	15.59
Subtotal	103.27
Tax	8.26
Dinner Total	111.53
Total Expenses	160.78



BIG PICTURE QUESTION

Explain at a high level how a large language model generates a response. What are some key limitations of LLMs that you would need to account for when building applications on top of them?

Take 30–60 seconds to think before sharing your answer. Record a strong answer in your ML Interview Prep Document.

Prompt Engineering: Core Techniques

Simple prompt:

Draft a plan for me to spend a weekend trip to Austin, Texas on a budget.

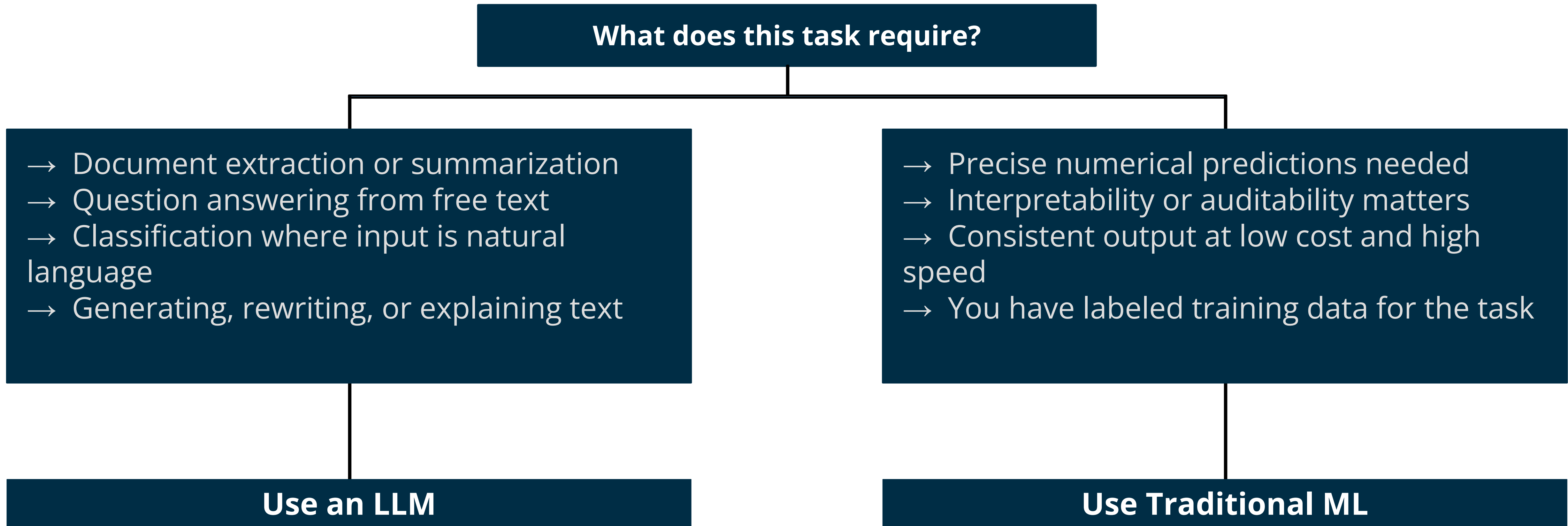
Reflection-Based Prompts:

Draft a plan for me to spend a weekend trip to Austin, Texas on a budget.
Draft: Create a first draft of the weekend plan.
Reflect: Evaluate the draft for gaps, unrealistic budgeting, or missed opportunities, and list critiques.
Refine: Rewrite the plan, addressing each critique.

Chain-of-Thought Prompts:

Give me a plan for a weekend trip to Austin, Texas on a budget. First, brainstorm all the different possible activities and options (free attractions, cheap eats, transportation choices, lodging alternatives, live music, outdoor spots, etc.). Then create a methodology to rate these options – propose a few different scoring approaches (e.g., cost vs. enjoyment, time efficiency, uniqueness), and pick the best one. Next, score all the potential activities using that method. Finally, use the highest-scoring activities to build the final weekend program.

LLM-Based vs. Traditional ML





BIG PICTURE QUESTION

When would you choose an LLM-based approach to solve a problem over a traditional ML model, and when would you choose the reverse? Give an example of each.

Take 30–60 seconds to think before sharing your answer. Record a strong answer in your ML Interview Prep Document.

Building Blocks of an AI Chatbot

Memory Management

Keeps conversation history within the context window.

- Trim and/or summarizes oldest information when history grows too long
- Persist key facts to a database for long-term recall
- Keeps cost down: every token in history is paid for on every call

Conversation History

The full record of chat turns passed back to the model on every API call.

- LLMs are stateless — they see only what you send
- The app must store and send history to create "memory"
- Each new chat turn appends to the list: system → user → assistant → ...

System Prompt

Developer-controlled message that defines persona and behavior before the user speaks.

- Sets tone, role, constraints, and knowledge scope
- Invisible to the user. The developer sets how the chatbot will respond but this isn't usually seen by the user
- "ChatGPT wrapper" startups are built on a compelling system prompt

The Model

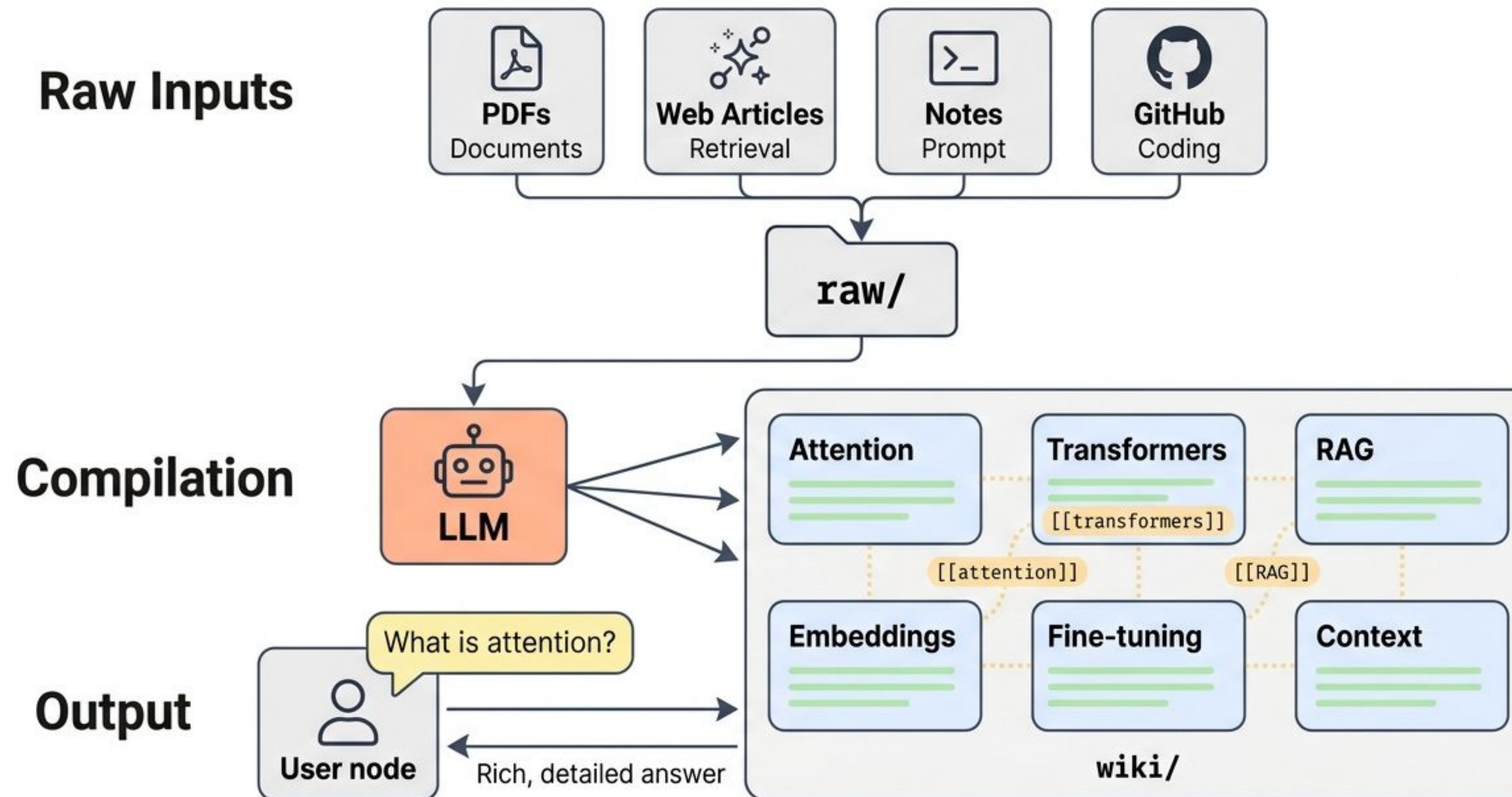
The LLM itself, accessed via API. Handles all language understanding and generation.

- GPT-4o, Claude, Gemini. You can swap between these models and more without changing your code much.
- Stateless: each API call is independent
- Outputs one token at a time; streaming sends them as produced

LLM Wiki's

LLM Wiki by Andrej Karpathy

data science dojo
— data science for everyone —



Compounding knowledge — answers get better over time.

LLM Wiki's: Personal Example

3. Resources + LLM Wiki

- Digital Bites
 - concepts
 - Intuitive Eating.md
 - Live Biofeedback.md
 - Mindful Eating.md
 - Mindfulness-Based Eating Awa...
 - Mindless Computing.md
 - Mukbang.md
 - Normative Consumption Cues....
 - Peripheral Intervention.md
 - Persuasive Systems Design.md
 - Portion Size Effect.md
 - Problematic Smartphone Use....
 - Screen as Companion.md
 - Screen-distracted Dining.md
 - Segmentation Cues.md
 - Skeating and Tele-dining.md
 - Transtheoretical Model.md
 - entities
 - raw
 - Blass on the road to obesity.pdf
 - BreathePulse.pdf
 - BreathTray.pdf
 - brightbeat.pdf
 - Capturing Eating Behavior fro...
 - Cognitive Control of eating.pdf
 - Consistency of eating rate, ora...
 - Counting bones.pdf
 - Digital behaviour change interv...
 - Digital_Commensality.pdf
 - Dining in the dark.pdf

TeX 01.introduction.tex
TeX 03.results.tex

1. Human Projects > Digital Bites (JMIR Formative) > sections > TeX 03.results.tex > Results > Participant Flo

```

\subsection{Quantitative Results}
\label{results-quant}

\subsubsection{Decreased food intake}
\label{results-quant-intake}

Supporting H1, Digital Bites significantly reduced the amount of food consumed (Cohen
$d_z$=0.66, 95\% CI 0.23-1.24, \textit{P}=0.007). Participants consumed an average of
255 grams during the intervention condition, compared with 305 grams in the control
condition: a reduction of 49 grams (16\%; 95\% CI 15-83 g), equivalent to
approximately 128 kcal (95\% CI 40-217 kcal) (Table~\ref{tab:quantitative-outcomes}).

\input{sections/latex-figures/tab-results}

Overall, 16 out of 21 participants (76\%) ate less during the intervention than in the
control condition, as shown in Figure~\ref{fig:distribution}. Participants who ate
more in the control condition generally also ate more in the intervention condition;
however, intake shifted downward across most participants. The regression line had a
slope below 1 ($Y=0.72X+36.14$, $R^2=0.64$, \textit{P}<0.001), indicating that
intervention intake increased with intake in the control condition, but at a reduced
rate. Figure~\ref{fig:regression} visualizes this pattern: points below the diagonal
line represent participants who ate less during the intervention condition. The size
of the intake reduction was not associated with dietary restraint (DEBQ-R; $r=-0.17$,
\textit{P}=.46) or BMI ($r=-0.07$, \textit{P}=.76, \textit{n}=20), aligning with prior
findings on distracted eating~\cite{robinson_eating_2013}.

\input{sections/latex-figures/fig-results-regression}

\subsubsection{Increased attention to and memory of the meal}

```

New Agent Claude Code Codex

Write RW discussion section

\subsection{Redesigning the Screen as a Mind...

I am trying to write this section. Could you help me by any chance? It would be ideal if you went through the RW in the LLM Wiki (possibly even the raw pdf's tbh), and told me what is discussable about our results, with potential implications to RW. I have no idea what the opportunities may be. We need to know.

Worked for 6m 24s

Yes—there is a real discussion contribution here. The strongest version is:

Digital Bites does not merely place feedback on the screen. It divides regulation between a persistent consumption trace that communicates *how much* and carefully timed checkpoints—chimes, content transitions, and possibly late-meal moments—that suggest *when* to reconsider eating, while leaving *what to do* to the diner.

That extends the related work in a defensible way.

What your results let you discuss

Result	Connection to related work	Defensible implication
13/21 found the overlay natural or unobtrusive; participants consulted it	Glanceable displays distinguish interpreting from capturing attention	Design the persistent bar and the event cues separately: the bar preserves state; the chime creates a

Multimodal LLMs



“Four score and seven years ago...”



[Some text output]



Attention Is All You Need

Ashish Vaswani*

Google Brain

avaswani@google.com

Noam Shazeer*

Google Brain

noam@google.com

Niki Parmar*

Google Research

nikip@google.com

Jakob Uszkoreit*

Google Research

uszko@google.com

Llion Jones*

Google Research

llion@google.com

Aidan N. Gomez*[†]

University of Toronto

aidan@cs.toronto.edu

Lukasz Kaiser*

Google Brain

lukaszkaiser@google.com

Illia Polosukhin*[‡]

illia.polosukhin@gmail.com

Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.8 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature. We show that the Transformer generalizes well to other tasks by applying it successfully to English constituency parsing both with large and limited training data.

[†]Equal contribution. Listing order is random. Jakob proposed replacing RNNs with self-attention and started the effort to evaluate this idea. Ashish, with Illia, designed and implemented the first Transformer models and has been crucially involved in every aspect of this work. Noam proposed scaled dot-product attention, multi-head attention and the parameter-free position representation and became the other person involved in nearly every detail. Niki designed, implemented, tuned and evaluated countless model variants in our original codebase and tensor2tensor. Llion also experimented with novel model variants, was responsible for our initial codebase, and efficient inference and visualization. Lukasz and Aidan spent countless long days designing various parts of and implementing tensor2tensor, replacing our earlier codebase, greatly improving results and massively accelerating our research.

[‡]Work performed while at Google Brain.

[‡]Work performed while at Google Research.

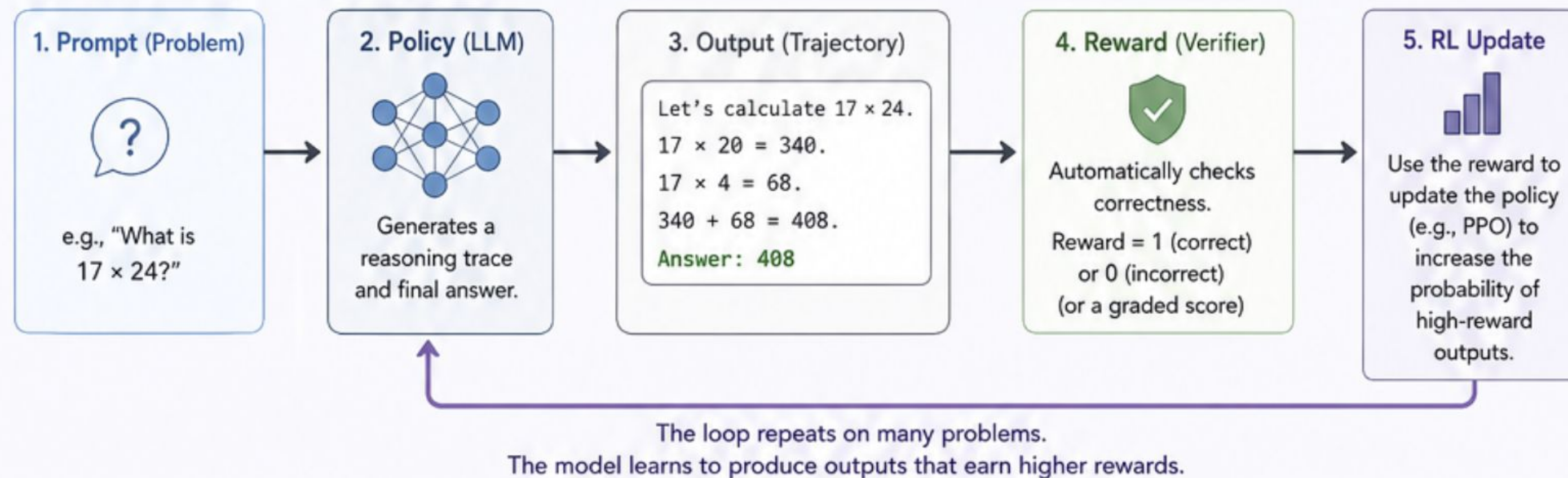
31st Conference on Neural Information Processing Systems (NIPS 2017), Long Beach, CA, USA.

gif from: <https://giphy.com/gifs/ufo-alien-meme-cat-fiWRwKU7MgXI750Kfk>

BREAK THROUGH TECH

RL Post-training for reasoning

1) HOW RL POST-TRAINING WORKS



2) WHAT IS A VERIFIABLE TASK?

A verifiable task has an objective answer that can be automatically checked by a program (the "verifier").
This enables reliable rewards for RL.

	 Math / Arithmetic	x^2 ... Algebra / Equations	 Code Generation	 Proof / Logic	☒ Multiple Choice / QA
Example Task	What is 137×86 ?	Solve for x : $2x + 3 = 11$	Write a function that returns the n th Fibonacci number.	Prove: In any group, the identity element is unique.	Which planet is known as the Red Planet? (A) Earth (B) Mars (C) Venus (D) Jupiter
Verifier Checks	Exact numeric equality to 11782	Check solution set $x = 4$	Run unit tests on hidden test cases	Check proof with a proof assistant (e.g., Lean, Coq)	
Reward Signal	1 if correct, else 0	1 if correct solution, else 0	1 if all tests pass, else 0 (or partial)	1 if proof validates, else 0	Exact match to correct option: (B)
Why it's Verifiable	Deterministic computation	Symbolic or numeric solver can verify	Tests deterministically check behavior	Formal verifier checks validity	Discrete set of possible answers

★ Key idea: Verifiers turn correctness into a machine-checkable reward, which RL can optimize.

RL Post-training for reasoning

3) WHY RL WORKS BETTER FOR CODING & MATH THAN FOR FREE-FORM TASKS

High Verifiability (e.g., Code, Math, Logic)

Clear right or wrong → reliable rewards → strong learning signal

Example: Code Generation

Model Output	<pre>def is_prime(n): if n < 2: return False for i in range(2, int(n**0.5)+1): if n % i == 0: return False return True</pre>
Verifier	Run unit tests on many cases (hidden from model): [1, 2, 3, 4, 5, 97, 101, 561, 1699, ...] All tests pass ✓
Reward	1 (high) Strong positive signal → model learns this pattern

Properties: deterministic evaluation, many test cases, precise correctness, sparse but trustworthy rewards.

RL Signal
Quality



High

vs.

Low



Low Verifiability (e.g., Essays, Explanations, Open-Ended)

Vague criteria → noisy rewards → weaker learning signal

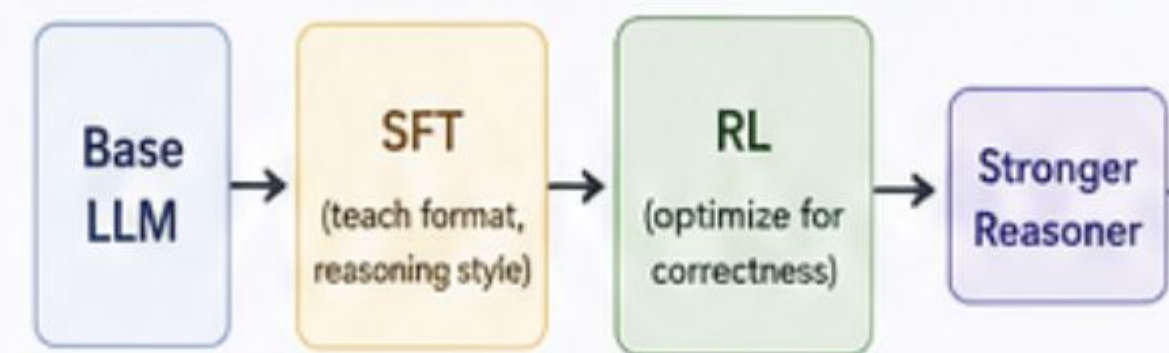
Example: Free-Form Explanation

Model Output	The Industrial Revolution was a period of rapid industrialization that brought many changes to society, including improvements in technology, transportation, and living standards for many people...
Verifier (LLM-as-Judge)	Rate helpfulness, correctness, and clarity on a 1–5 scale? Maybe 3/5. (But another judge might give 4/5.)
Reward	0.3 (normalized) Noisy, subjective signal → slower, less reliable learning

Challenges: subjective evaluation, multiple valid answers, style/length biases, hard to detect subtle errors.

4) PRACTICAL IMPLICATIONS

- ✓ RL post-training specializes the model in precise reasoning.
- ✓ Best used with large amounts of verifiable problems.
- ✓ Often combined with supervised fine-tuning (SFT) first, then RL for further gains.
- ✓ Techniques: PPO (common), GRPO, RLOO, DPO (preference optimization as an alternative).



5) TAKEAWAY

- ★ RL + verifiable rewards = powerful feedback loop. When correctness is easy to check (like in code and math), the model learns to reason and solve problems much more reliably.



BIG PICTURE QUESTION

What are the key building blocks of an AI chatbot built on top of an LLM? Walk through what each one contributes to the user experience.

Take 30–60 seconds to think before sharing your answer. Record a strong answer in your ML Interview Prep Document.

Share AI uses you are excited about

Overview

1. Let's break into rooms of 3-4, and discuss the AI uses in our daily lives, that we are excited about.
2. Document these discussions with words or images, in this presentation (title with your room number).

Example: I recently created a Claude-automated todo-list. Whenever I create a todo in my application, it tries to find all available resources for me to complete a task, and document it in the todo note. It's made my life much easier:)



I also created a speech transcription shortcut in 20 minutes with Codex, canceling my OtterAI/SuperWhisper subscriptions



Questions?

Any final questions?

30 MIN · 2 PARTS

Business Understanding Session

PART 1 · 15 MIN Business Understanding Discussion

Discuss the Applied Business Questions in groups of 3.

PART 2 · 15 MIN Business Alignment

Whole group aligns before coding begins.

Business Brief + Questions

The Business Context

SpendLens is a fintech startup that builds AI-powered expense management tools. Their new client -- a property management company overseeing 40 rental properties -- needs to replace manual data entry with an automated pipeline that extracts structured data from vendor expense documents. You are a junior ML engineer on the Document Intelligence team, tasked with building and testing that pipeline using a vision-capable LLM.

Discuss in your groups:

1

SpendLens is building a pipeline to extract structured data from expense document images using a LLM that uses a tool for extracting information from images. Consider writing a prompt that tells the LLM to extract the text from an image. What information would you include in that prompt?

2

Once this product is built out, SpendLens will consider offering this pipeline as a product to more clients. What would a client need to know before trusting it to handle their expense documents automatically? What failure modes would concern you most, and what would you want to have in place before handing it off?

Business Alignment - Let's Discuss

The Business Context

SpendLens is a fintech startup that builds AI-powered expense management tools. Their new client -- a property management company overseeing 40 rental properties -- needs to replace manual data entry with an automated pipeline that extracts structured data from vendor expense documents. You are a junior ML engineer on the Document Intelligence team, tasked with building and testing that pipeline using a vision-capable LLM.

Discuss in your groups:

1

SpendLens is building a pipeline to extract structured data from expense document images using a LLM that uses a tool for extracting information from images. Consider writing a prompt that tells the LLM to extract the text from an image. What information would you include in that prompt?

2

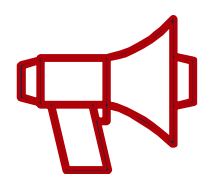
Once this product is built out, SpendLens will consider offering this pipeline as a product to more clients. What would a client need to know before trusting it to handle their expense documents automatically? What failure modes would concern you most, and what would you want to have in place before handing it off?

70
MIN

Coding Working Session

Lab 7: In this lab, you will:

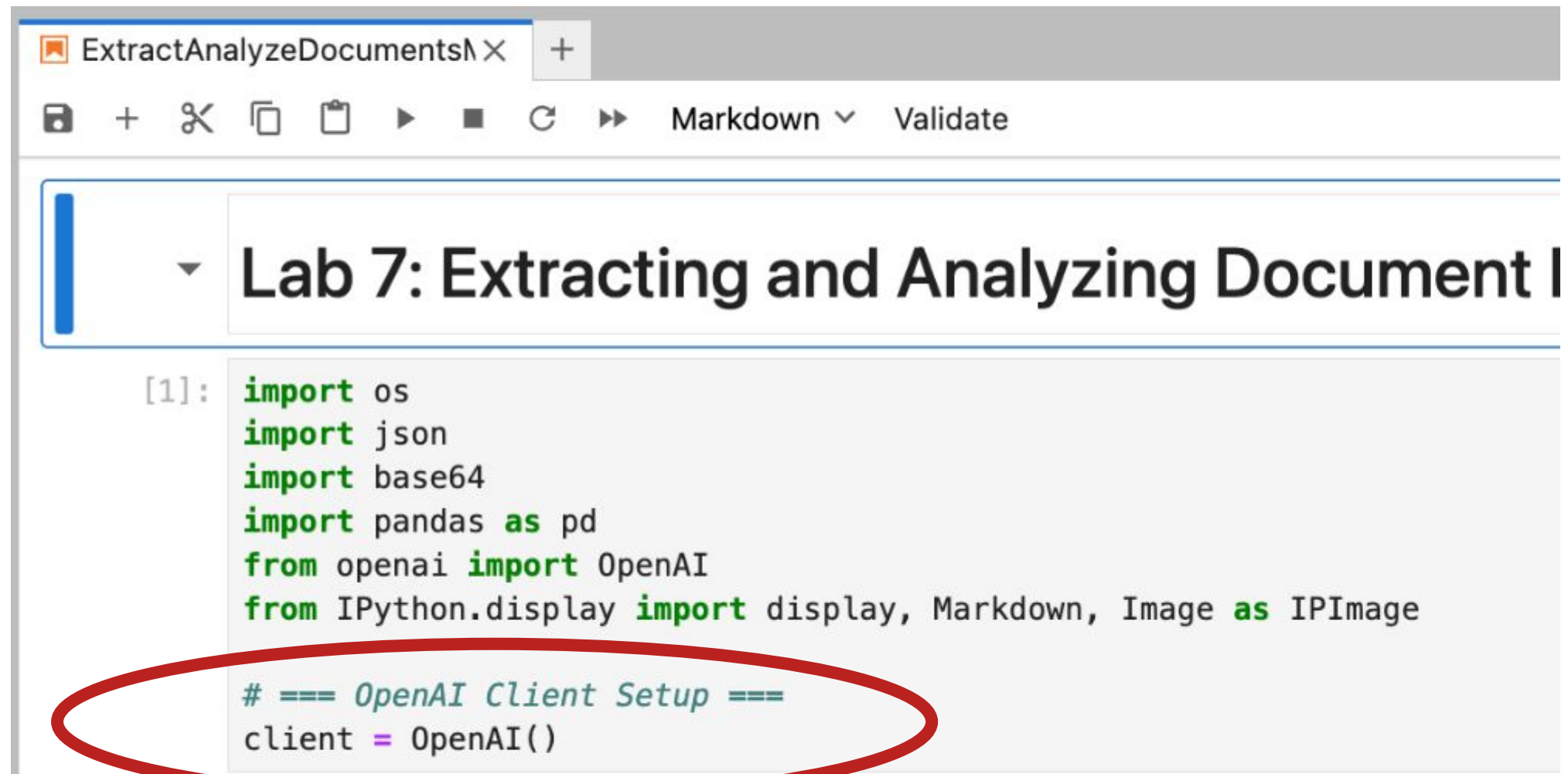
- Load expense document images and convert them to base64 for API use
- Write an extraction prompt and extract structured data (vendor, date, items, totals, category) from each document
- Parse JSON responses, build a summary DataFrame, and run a self-reflection validation pass
- Use the LLM to analyze spending patterns across the batch and evaluate the pipeline's accuracy and business risk



You can work with other fellows on the lab and you can use AI tools in 'tutor mode' for support. Regardless of the help you use, you should be able to explain every line of code you write.

Before you begin...

In this lab and all upcoming labs, you will be interacting with an API key.



```
[1]: import os
import json
import base64
import pandas as pd
from openai import OpenAI
from IPython.display import display, Markdown, Image as IPIImage

# === OpenAI Client Setup ===
client = OpenAI()
```

There is a budget associated with this API key. As long as you are completing the assignments as designed, you should not hit this budget limit. You don't need to do anything different, and the key is hidden from you, so you don't have to worry about it.

If you hit the limit, you will get an error message telling you to contact your instructor. Send me a note if you get this error and I will reset your system so you can continue working.

```
AuthenticationError: Error code: 401 - {'error': {'code': '401', 'message': "[BricksLLM] You've reached your AI usage limit for this course. To continue, please contact your instructor or TA.", 'type': ''}}
```

Important: Do NOT wait until the last minute to complete these labs. If you hit the budget limit close to the deadline, I may not be available to reset it in time for you to submit.

CODE SHARE

20 MIN MARK · 40 MIN MARK · 60 MIN MARK

You will be in breakout rooms in groups of 3 for the coding session. Three times during the lab, we will pause for a Code Share where one fellow walks your breakout room through their progress.



Share your screen

Walk your group through where you are and what you're working through.



Peers respond

Listen, ask questions, offer help or a different approach.



Roles rotate

Every fellow shares once across all 3 rounds. This is round 1 of 3.

10
MIN

Group Retro + Closing

What Did We Learn?

How did you use AI while working on this lab?

What worked well in the coding session? What would you have done differently next time?
Where did you get stuck?

Before you go —

Complete the Exit Ticket

Survey Link (pasted in Discord):

https://breakthroughtech.tfaforms.net/wf91mPA?tfa_5=Weekly-Fellow-Survey-MLF

Announcements + Deadlines

[Add deadline reminders and announcements here]